تمرین امتیازی شماره 1

عنوان : Elgamal Signature

درس : امنیت سخت افزار

استاد : دکتر سیامک محمدی

دستیار آموزشی : علیرضا رمضانی

نیمسال دوم تحصیلی سال 1406-1405

مقدمه(Introduction)

امنیت سازوکارهای رمزنگاری در سامانه های سخت افزاری، به ویژه در HSM ها و Secure Elementها، نه تنها به استحکام الگوریتم های رمزنگاری پایه وابسته است، بلکه به نحوه پیاده سازی عملی آنها نیز وابستگی عمیق دارد . تاریخچه حملات عملی نشان می دهد که بسیاری از شکستهای امنیتی نه به دلیل ضعف نظری الگوریتمها، بلکه به علت خطاهای پیاده سازی، مدیریت نادرست تصادفی سازی، و نقض فرضیات امنیتی رخ دادهاند. یکی از مهمترین نمونه های این مسئله، امضای دیجیتال ElGamalاست؛ الگوریتمی که امنیت آن به طور مستقیم به غیرقابل پیشبینی بودن مقدار تصادفی (k) nonce در هر امضا وابسته است. در سناریوهای سخت افزاری، این مقدار معمولاً درون دستگاه تولید شده و هرگز از آن خارج نمیشود. با این حال، اگر به هر دلیل(ضعفRNG ، خطای طراحی، یا بهینه سازی نادرست) از یک nonce یکسان برای امضای دو پیام متفاوت استفاده شود، کلید خصوصی امضا میتواند بهصورت محاسباتی و قطعی بازیابی شود؛ حتی بدون نیاز به شکستن فرض لگاریتم گسسته. این تمرین با هدف درک عمیق آسیب پذیری های پیاده سازی امضاهای رمزنگاری در سخت افزار امن طراحی شده است. در این تکلیف، یک پیاده سازی معیوب از امضای هش شده ی ElGamalدر قالب فایل `buggy_elgamal.py` بررسی میشود که رفتار آن مشابه یک HSM یا Secure Element در نظر گرفته شده است؛ بهگونه ای که کلید خصوصی هرگز افشا نمیشود، اما دستگاه به اشتباه از nonceتکراری برای امضای هش های مختلف استفاده میکند.

نحوه عملکرد امضای ElGamal

طرح امضای دیجیتال ElGamal یک الگوریتم امضای کلید عمومی است که بر اساس گروه ضربی به پیمانه a عدد اول و سختی فرضی مسئله لگاریتم گسسته(DLP) بنا شده است. در عمل، امضاهای ElGamal به جای خود پیام، بر روی هش پیام اعمال میشوند که این امر کارایی را بهبود میبخشد و ورودیهای با طول ثابت را تضمین میکند.

1.پارامترها و کلیدهای سیستم

g مولد گروه ضربی $Zp*$ باشد ، $x\in[1,p-2]$ کلید امضای خصوصی است ، $y=g^x \bmod p$ کلید تأیید عمومی باشد ، پارامترهای عمومی عبارتند از(p,g,y) ، در حالی که کلید خصوصی x در داخل دستگاه امضا (مثلاً یک HSM یا عنصر امن) مخفی نگه داشته می شود.

۲ .فرایند امضا

برای امضای یک پیام m ، امضاکننده ابتدا هش آن را محاسبه میکند: $h=\text{hash}(m), h\in[0,p-2]$ ، سپس امضا به مرحله ۱ می رسد: انتخاب یک nonce تصادفی انتخاب یک مقدار تصادفی جدید: صورت زیر تولید می شود:

$k\in[1,p-2]$ به طوری که $\gcd(k,p-1)=1$.

این nonce باید برای هر امضا منحصر به فرد و غیرقابل پیشبینی باشد.

مرحله ۲: محاسبه اولین مؤلفه امضا

$$r=g^k \bmod p$$

مرحله ۳: محاسبه مؤلفه دوم امضا

$$s=(k^{-1})\cdot(h-xr)\bmod(p-1)$$

امضای elgamal حاصل این جفت است:(r,s)

۳ .تأیید امضا

برای تأیید امضا(r،s) روی هش h، تأییدکننده بررسی میکند که آیا:

$g^h \equiv y^r \cdot r^s \pmod{p}$

• اگر تساوی برقرار باشد، امضا معتبر است.

• در غیر این صورت، امضا رد میشود.

4. صحت طرح

از آنجا که:

$y=g^x$ و $r=g^k$ ، ما داریم :

$y^r \cdot r^s = g^{xr} \cdot g^{ks} = g^{(xr+ks)}$ طبق s:

تعریف :

$$s \equiv (k^{-1})(h - xr)(mod\ p-1) \Rightarrow ks \equiv h - xr \pmod{p-1}$$

بنابراین :

$xr+ks \equiv h \pmod{p-1}$

که به این معنی است:

$y^r \cdot r^s \equiv g^h \pmod{p}$

این ثابت میکند که امضاهای تولید شده صادقانه همیشه به درستی تأیید میشوند.

۵.قش nonce و پیامدهای امنیتی :

امنیت امضای ElGamal بر دو فرض اساسی متکی است:

سختی مسئله لگاریتم گسسته،تازه بودن و محرمانه بودن نانس k .

اگر از همان nonce k برای امضای دو هش مختلف h1 و h2 استفاده شود:

مقدار $r=g^{\wedge}k \bmod p$ در هر دو امضا یکسان خواهد بود، این منجر به دو معادله خطی با مجهولات یکسان k و x میشود. در نتیجه یک مهاجم میتواند به طور موثر k را بازیابی کند و سپس کلید خصوصی x را محاسبه کند.

بنابراین، استفاده مجدد از nonce امنیت امضاهای ElGamal را به طور کامل از بین میبرد، حتی اگر کلید خصوصی هرگز سخت افزار امن را ترک نکند.

سوال 1) (20 نمره)

شما می بایست به صورت دستی اثبات کنید که می توان secret_key و nonce را بازیابی کرد.

نکات تمرین :

پارامترهای عمومی (p, g, y) می باشند و x کلید خصوصی امضا می باشد .دستگاه(HSM or secure element) هش ها را در بازه [0, p−2] امضا می کند.

در فایل پایتونی buggy_elgamal.py،keygen() پارامترهای عمومی بازمی گرداند و x درون دستگاه می ماند ، تابع sign_hash(h, p, g) -> (r, s) هش را امضا می کند .

verify_hash(h, (r, s), p, g, y) -> bool : امضای روی هش h را درست سنجی می کند که این امضا معتبر است یا خیر.

سوال 2) با استفاده از کد پایتون ، مقادیر x, k را بازیابی کنید (۳۰ نمره)

راهنما :

buggy_elgamal،modinv, sign_hash, keygen, verify_hash این موارد را شما می توانید همراه کدی که فراهم می کنید برای بدست آوردن k , x استفاده کنید.

لازم است در گزارش خود هر بخش از کد زده شده را توضیح دهید و اسکرین شات خروجی هایی که بدست می آورید را در گزارش خود بیاورید .

سوال 3) در این قسمت با استفاده از x بازیابی شده باید جعل امضای elgamal را انجام دهید ؟ (30 نمره)

در این قسمت نیز بخش های مختلف کد زده شده را توضیح دهید و با ذکر دلیل توضیح دهید چطور این کار ممکن است ؟

بخش پرسش های تمرین

(20 نمره)

سوال 1) چرا استفاده مجدد مقدار تصادفی (nonce)امنیت امضاهای ElGamal را به خطر می اندازد ؟

سوال 2) تفاوت بین جعل یک امضای معتبر و محاسبه جفت امضای جعلی (r, s) چیست؟ چرا جعل باید هدف r1, s1 (یا r2, s2) باشد

سوال 3)] چرا امضای Elgamal از مد p−1 برای s استفاده می کند به جای مد p ؟

سوال 4)

چه فرض اساسی امنیت امضای ElGamal را تضمین میکند؟

.

مواردی که دانشجویان باید تحویل دهند :

1) حل سوال 1 به طور دستی

2) فایل کد پایتونی attacker_elgamal.py به همراه توضیحات نحوه عملکرد کد

3) یک فایل گزارش pdf از پاسخ کل سوالات تمرین (توضیح بخش های تئوری + بخش های عملی)